

Task 2 - VSDSquadron Mini Board Bring-Up, GPIO & UART Validation

Deadline: 4 days

Applies to: All enrolled participants

Objective

The objective of **Task-2** is to transition from *host-based firmware development* (Task-1) to **real hardware bring-up on the VSDSquadron Mini RISC-V board**.

In this task, you will:

- Power up and flash firmware on the actual board
- Validate UART communication
- Map physical board pins to firmware GPIO definitions
- Build a **minimal GPIO + UART firmware layer**
- Demonstrate correct hardware–software interaction using evidence

This task ensures you can **boot, flash, debug, and control real RISC-V hardware**, which is a fundamental industry skill.

Pre-Requisites

Before starting Task-2, you must have:

- Completed **VSDSQMiniTask-1**
- Received the **VSDSquadron Mini board**
- Read the **VSDSquadron Mini Board Datasheet / Flashing Guide**
- A working development environment (toolchain, flasher, serial terminal)

Important:

The **board datasheet and flashing instructions are your starter kit** for Task-2.

No additional template will be provided.

Task-2 Scope

You are required to **build and demonstrate** the following on the VSDSquadron Mini board:

1. Board Bring-Up & Firmware Flashing

- Successfully flash a firmware binary onto the board
- Ensure the board boots correctly after flashing
- Capture UART output after reset

2. UART Validation (Mandatory)

Implement UART output such that:

- A clear startup message is printed after reset
- Output includes:
 - Board name
 - Firmware version (hardcoded string is fine)
 - A continuously increasing counter OR periodic message

Minimum requirement:

- UART log must show **at least 10 consecutive lines**
- Output must be readable on a serial terminal

3. GPIO Pin Mapping & Validation (Mandatory)

You must demonstrate **correct GPIO control** using **at least one physical pin**.

This includes:

- Selecting one GPIO pin available on the board
- Identifying:
 - **Physical pin label** (as printed on the board / silkscreen)
 - **Firmware GPIO number** (as per datasheet / board header mapping)
- Writing firmware to:
 - Configure the pin as OUTPUT
 - Toggle it (LED blink or measurable output)

 The GPIO number used in firmware **must match the datasheet mapping**.
Do not invent your own numbering.

4. Firmware Structure Requirement (Important)

Your firmware **must NOT** directly manipulate hardware registers inside `main.c`.

Instead:

- Create a **small GPIO API layer**
- Use function calls such as:
 - `gpio_init()`
 - `gpio_set()`
 - `gpio_clear()`
 - `gpio_toggle()` (optional)

This mirrors **real embedded firmware architecture** used in industry.

Expected Repository Structure

Inside your fork of the internship repository, create the following:

```
vsdsquadron-mini-core/  
├── task2/  
│   ├── submission/  
│   │   ├── main.c  
│   │   ├── gpio.c  
│   │   ├── gpio.h  
│   │   ├── evidence.md  
│   │   └── README.md
```

File Expectations

main.c

- Application logic only
- Calls GPIO and UART functions
- No direct register access

gpio.c / gpio.h

- GPIO abstraction functions
- Datasheet-based pin definitions

README.md

Must contain:

- Brief explanation of what you implemented
- GPIO pin chosen and why
- UART message description
- How to build and flash (short steps)

evidence.md (Mandatory)

Must include:

- 1. UART Evidence**
 - Screenshot and video showing 10+ lines of UART output
- 2. GPIO Evidence**
 - Photo and a short video clearly showing:
 - The physical board
 - The pin being toggled (LED / probe / visible signal)
 - Mention:
 - Physical pin label
 - Firmware GPIO number
- 3. Short explanation**
 - How you verified correct behavior

Evaluation Criteria (Pass/Fail)

You will **PASS Task-2** only if **all** of the following are true:

- Board boots and firmware flashes successfully
- UART prints readable output (10+ lines)
- GPIO pin toggles correctly on real hardware
- GPIO number matches datasheet mapping
- Firmware uses an API layer (no raw register access in `main.c`)
- Repository structure is correct
- `evidence.md` clearly proves hardware execution

Learning Outcome

After completing Task-2, you should be confident in:

- Bringing up a real RISC-V board
- Flashing and debugging firmware
- Using UART as a primary debug interface
- Mapping physical hardware to firmware definitions
- Writing clean, layered embedded firmware

This task forms the **foundation for building reusable firmware libraries and IPs** in later stages of the internship.